CSF2753 Lab Class

Filippos Pantekis

CSF2753 Lab Class 5 21/10/2026

Parameter Passing and Response Building

In this lab, you will implement a simple RESTful web service using Spring Boot that allows clients to query and modify a reduced version of the UCI Adult Dataset.

The aim of this lab is to focus on implementing:

- Parameter passing via HTTP requests (`@RequestParam`, `@PathVariable`, `@RequestHeader`).
- Returning informative responses using `ResponseEntity` for more flexible HTTP status handling.
- Performing CRUD operations on a dataset.

To start, there is a zipped `Spring Boot` project on Canvas, under:

`Modules → Week 5 → Lab`

Within the project you will find:

- a `data` directory containing the `adult.csv` dataset, located under the `resources` folder.
- A package structure separating models, controllers and services.
- An `Adult` class defining the relevant fields, constructor and accessor methods.
- An `AdultReaderService` responsible for reading the `csv` file and returning a `HashMap` containing unique ID numbers and their corresponding Adult objects.

Finally, since the in-memory container used for the adult dataset is a `HashMap`, a quick guide to using this object has been appended to the end of this lab sheet.

☐ Task 5.1

Create a simple `AdultController` class that:

- Loads the dataset in memory when the application starts.
- Allows clients to view the entire dataset as a JSON response.
- Allows clients to customise this response by specifying a `salaryClass` in their request.

☐ Task 5.2

Modify the controller to support **Create** operations:

- Add a simple endpoint that allows clients to create a new `Adult` entry in the dataset.
- The client should be able to pass the relevant data via the HTTP request body.
- Ensure the service returns the created instance as proof of construction.

- Consider whether the current code separation allows this to be done sensibly. If it should be changed, how should it be changed?

☐ Task 5.3

Adjust the controller class to protect against unauthorized data removal:

- Add an endpoint that allows clients to delete `Adult` entries that fall within a client-defined `age` range.
- Ensure this request only succeeds if the client has an `Authorization` header that matches some `token` held by the server. (The token can be any value you want it to be).

☐ Task 5.4

Adjust your controller class methods to always return a sensibly configured `ResponseEntity`. Your implementation should consider:

- How many logical branches does the method contain?
- Which HTTP codes are most informative for the client, given that branch?
- Does the response include an HTTP body? If so, how is it wrapped by the `ResponseEntity`?

Please turn over ☞

Java HashMap Cheat Sheet

You undoubtedly remember all this from CSF1702, but a small recap doesn't hurt :-)

Remember: Hashmaps are **fast** when used well. The key of the map is unique, and operations using the key (getting/removing/containsKey) are supper fast.

Creating and Initialising

```
HashMap<String, Integer> scores = new HashMap<>();

//Associate the key (String) to a value (Integer).
scores.put("Alice", 85);
scores.put("Bob", 92);
scores.put("Charlie", 78);
```

Accessing Elements

```
//get() retrieves the value associated with a key
int aliceScore = scores.get("Alice");  // returns 85

//If a key or value exists can be checked like so:
boolean hasBob = scores.containsKey("Bob");
boolean hasScore92 = scores.containsValue(92);
```

Updating Elements

```
//Running this again, will replace the current value assigned to
// "Alice" with 90.
scores.put("Alice", 90);
```

Removing Elements

```
//Remove a key-value pair given the key.
scores.remove("Bob");
```

Iterating Over a HashMap

```
//Looping over all keys in the map
for (String name : scores.keySet()) {
        System.out.println(name);
}
//Looping over every value in the map
for (int score : scores.values()) {
        System.out.println(score);
}
//Looping over every entry (i.e., key-value pare)
for (Map.Entry<String, Integer> en : scores.entrySet()) {
        System.out.println(en.getKey() + ": " + en.getValue());
}
```